



THE UNIVERSITY OF
SYDNEY

Room Number _____

Seat Number _____

Student Number _____

ANONYMOUSLY MARKED

(Please do not write your name on this exam paper)

CONFIDENTIAL EXAM PAPER

This paper is not to be removed from the exam venue

Computer Science

EXAMINATION

Semester 1 - Mock, 2026

COMP2017 Systems Programming

For Examiner Use Only

EXAM WRITING TIME: 2 hours

READING TIME: 10 minutes

EXAM CONDITIONS:

This is a RESTRICTED OPEN book exam - specified materials permitted

MATERIALS PERMITTED IN THE EXAM VENUE:

(No electronic aids are permitted e.g. laptops, phones)

Bilingual dictionary

Calculator - non-programmable

One A4 sheet of handwritten and/or typed notes double-sided

Formula sheet (provided in the exam paper by unit coordinator)

Q	Mark
Q1	/10
Q2	/10
Q3	/10
Q4	/20
Q5	/20
Q6	/30
Total	/100

MATERIALS TO BE SUPPLIED TO STUDENTS:

Blank scratch paper (1 sheet; more may be provided upon student request)

INSTRUCTIONS TO STUDENTS:

Do not turn over this page until you are told!

- Answer all questions in the spaces provided in this booklet.
- Write your final answers in black or blue ink, not pencil. Take care to write clearly and legibly. Drawing diagrams using pencil is permitted.
- Please write your student number at the top of every odd numbered page.
- Do not tear or remove any pages from the booklet. Staff are not responsible for missing pages in this case. If you find there are missing pages, please alert the examination supervisors.
- State clearly on empty pages which question, or part of the question you are completing. There are extra working pages at the back of this question booklet.
- Return this booklet, all scratch paper and your A4 sheet of notes.

Please tick the box to confirm that your examination paper is complete. ☐

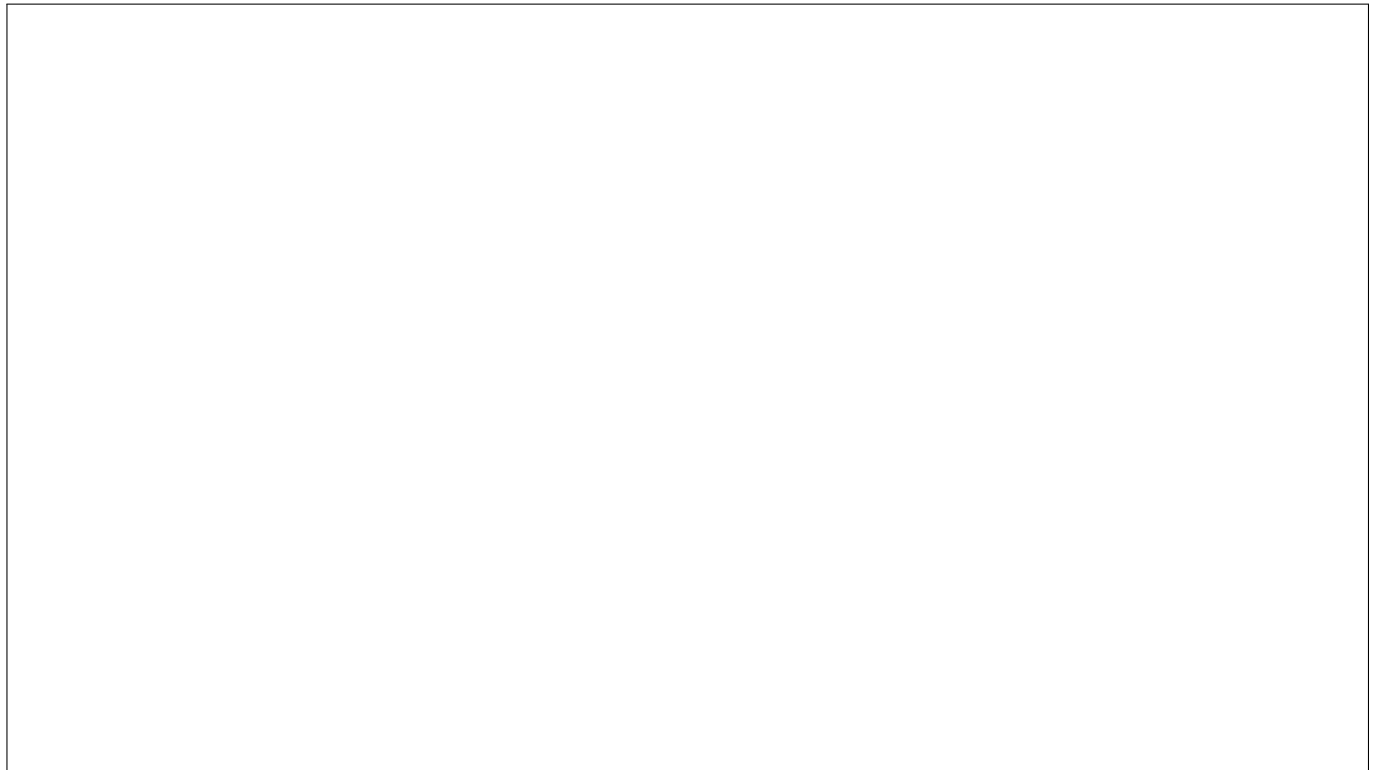
Question 1 Simple C (10 marks)**Q1(a) 5 marks.**

Write a C program that calculates the average of student marks if student marks are provided on each line within the file `stu_marks.csv`.

Sample Input:

```
77
54
42
99
```

You may assume that the file exists and contains integer values.

**Q1(b) 5 marks.**

Describe how you would modify your code if the file contained the student ID and mark separated by a comma on each line.

Sample Input:

```
212345,77
100203,54
131265,42
375463,99
```



Question 2 Data structures (10 marks)

```

1 struct vehicle {
2     uint32_t weight;
3     union {
4         struct {
5             uint32_t passengers;
6             uint32_t colour;
7         } car;
8         struct {
9             uint32_t seated_passengers;
10            uint8_t wheels;
11            uint32_t standing_passengers;
12        } bus;
13    } vehicle;
14    char license;
15 };
16
17 struct vehicle vehicles[2];

```

Using the struct definition and declaration above, determine the memory offset (in bytes) of each field from the start of a declared `struct vehicle`.

```
vehicles[0].weight
```

```
vehicles[0].vehicle.car.passengers
```

```
vehicles[0].vehicle.car.colour
```

```
vehicles[0].vehicle.bus.seated_passengers
```

```
vehicles[0].vehicle.bus.wheels
```

```
vehicles[0].vehicle.bus.standing_passengers
```

```
vehicles[0].license
```

```
vehicles[1].weight
```

[illegible]

Question 3 Security (10 marks)

Q3(a) 5 marks - Explain the difference between static libraries and shared objects.

Q3(b) 5 marks - Identify and explain the security concerns when using shared objects instead of static libraries.

Question 4 Concurrency (20 marks)

Peter's pizzeria, a new pizza restaurant on campus, has become very successful. Peter wishes to increase the throughput of his restaurant by employing multiple staff to serve multiple customers. However, customers keep receiving the wrong orders and the stock of ingredients is difficult to track.

```

1 #define MAX_TOPPINGS 3
2 #define MAX_ORDERS 10
3
4 enum toppings { TOMATO, MUSHROOM, PEPPERONI, CAPSICUM, DUCK, CHEESE, NTOPPINGS};
5
6 typedef struct { // You may modify this struct
7     enum toppings toppings[MAX_TOPPINGS];
8     int ntoppings;
9     int success;
10    int complete;
11 } order_t;
12
13 int stock[NTOPPINGS];
14 order_t* orders[MAX_ORDERS];
15 int order_head = 0;
16 int order_tail = 0;
17 // Declare additional variables as needed
18
19 // You may not modify this function
20 void prepare_pizza(order_t* order) {
21     order->success = 1;
22     // Check stock
23     for (int i = 0; i < MAX_TOPPINGS; i++){
24         if (stock[order->toppings[i]] == 0) {
25             order->success = 0;
26             goto prepare_pizza_complete;
27         }
28     }
29     // Adjust stock
30     for (int i = 0; i < MAX_TOPPINGS; i++) stock[order->toppings[i]]--;
31     prepare_pizza_complete:
32     order->complete = 1;
33 }
34
35 // You may modify the code below
36 void* staff_fn(void*) {
37     while(1) {
38         order_t* order = orders[order_head];
39         order_head = (order_head + 1) % MAX_ORDERS;
40         prepare_pizza(order);
41     }
42     return NULL;
43 }
44
45 int customer_order(enum toppings *toppings, int ntoppings) {
46     order_t order = {.ntoppings = ntoppings};
47     memcpy(&order.toppings, toppings, sizeof(*toppings) * ntoppings);
48     order.complete = order.success = 0;
49     orders[order_tail] = &order;
50     order_tail = (order_tail + 1) % MAX_ORDERS;
51     while(order.complete == 0);
52     return order.success;
53 }
54
55 void pizza_shop_init(void) {
56     for(int i = 0; i < NTOPPINGS; i++) stock[i] = 100;
57 }

```

Consider the code above that simulates Peter's pizzeria, where one or more customers call `customer_order` concurrently with an array of requested toppings and the number of requested toppings. One or more staff threads are created with `staff_fn` as the entry point.

Note that customers can request toppings in any order and may request additional servings of any ingredient. For example, a customer might order the following.

```
1    enum toppings toppings [] = {CHEESE, CHEESE, DUCK};
2    customer_order(&toppings, 3);
```

Rewrite `staff_fn` and `customer_order`, adding any synchronisation required to avoid all race conditions and all busy waiting. You may not modify `prepare_pizza` as Peter is quite pedantic about how a pizza should be prepared.

Pizza's should be prepared in parallel where possible (when staff do not require the same ingredients).

You may add additional global variables if required and/or perform any additional initialisation in `pizza_shop_init`.

Extra space for question Q4.

Question 5 Programming (20 marks)

Question 5(a) (15 marks)

The code below performs a **string** search, returning up to **max_offsets** offsets in the **haystack** string at which the **needle** string was found.

```

1 int search(const char* needle, const char* haystack, int* offsets, int max_offsets) {
2     int needle_len = strlen(needle);
3     int haystack_len = strlen(haystack);
4     int idx = 0;
5     for (int i = 0; i <= haystack_len - needle_len; i++){
6         int j;
7         for (j = 0; j < needle_len; j++) {
8             if (haystack[i + j] != needle[j]) {
9                 break;
10            }
11        }
12        if (j == needle_len) {
13            offsets[idx++] = i;
14            if (idx == max_offsets) {
15                break;
16            }
17        }
18    }
19    return idx;
20 }
```

Write C code for a parallel implementation of the above code that uses 10 pthreads to search the **haystack** in parallel. There are no requirements on the order in which offsets are reported.

Extra space for question Q5(a).

Extra space for question Q5(a).

Question 5(b) (5 marks)

What would be the advantages and/or disadvantages of using processes with fork instead of pthreads in this search problem? Discuss differences in shared spaces of processes and threads here.

Question 6 Programming (30 marks)

Write C code for a client-server chat program backend.

The server supports a fixed number of clients. The number of clients is passed as the first command line argument.

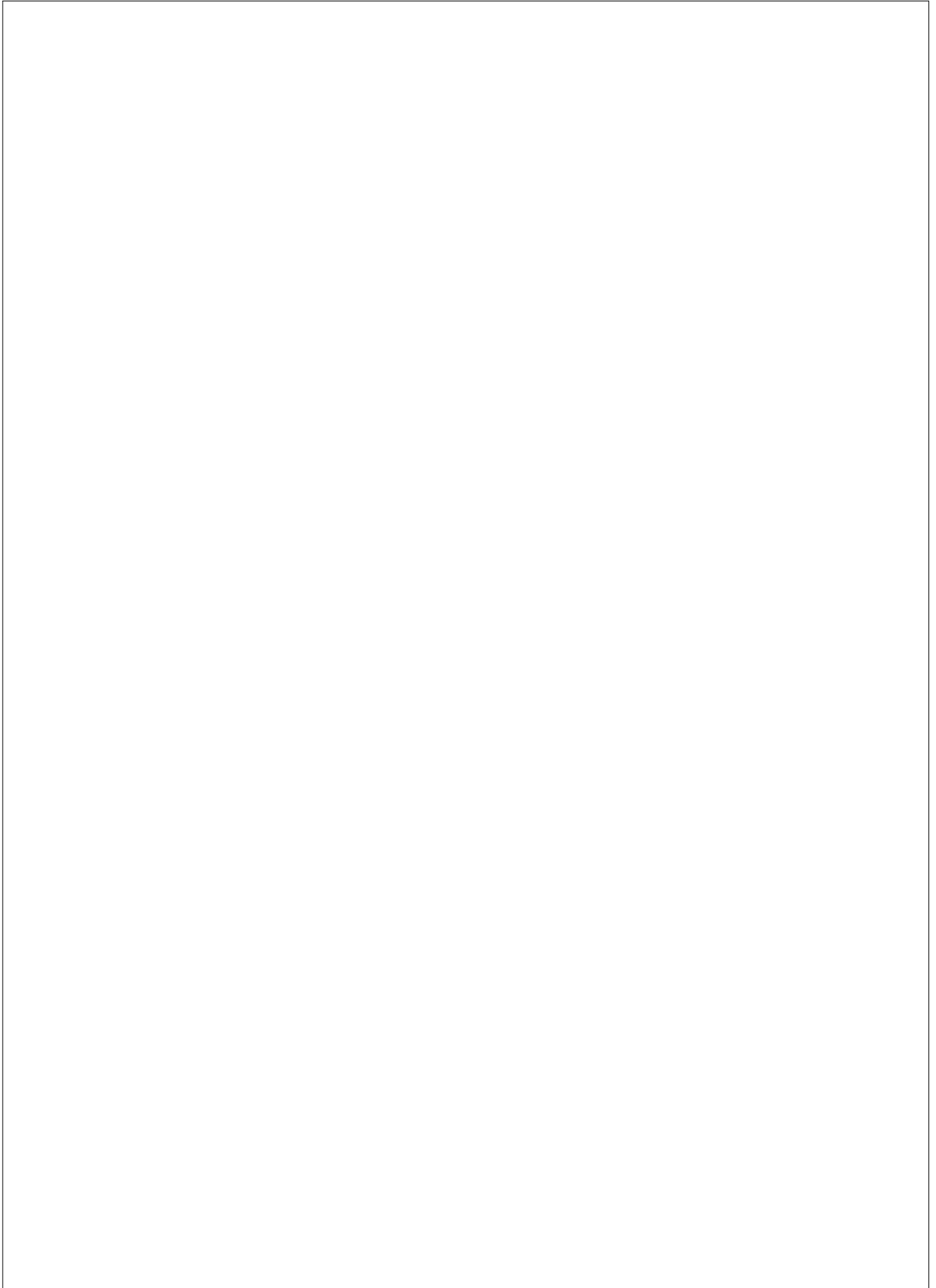
The client code is provided as an independent binary file that exists in the current working directory. The name of the binary is “client”. The server must create the required number of client processes to manage client input and output. You can assume that the “client” file exists and is complete.

Communication between client and server is via 2 anonymous pipes. The first pipe is used for communication from server to client while the second is used for communication between client and server. The *fd* associated with these pipes is passed as the first and second command line arguments to “client”. All communication must go through the server – the only fds open in a client should be *stdin*, *stdout*, *stderr* and the 2 anonymous pipes used to communicate between server and client.

For clients to maintain a history of all messages posted in chat, each message posted by a user is sent to the server via the provided anonymous pipe. The server must then broadcast that message to all other clients.

The server should terminate after all client-to-server pipes are closed by the client. If the server receives the signal SIGTERM, the server should terminate all clients by sending the SIGTERM signal to each client before the server terminates.

Extra space for question Q6.

A large, empty rectangular box with a thin black border, intended for providing an answer to question Q6. It occupies the majority of the page area below the header and the initial instruction.

Extra space for question Q6.

END OF EXAMINATION

End of examination – extra space

COMP2017; Write your SID:

page 17 of 26

End of examination – extra space

End of examination – extra space

COMP2017; Write your SID:

page 19 of 26

End of examination – extra space

Common types: `int`, `float`, `short`, `char`

Common escape characters: `\n` `\t` `\'` `\"` `\\` `\0`

Array notation: `[ ]` * **Address operator:** `&`

Keywords

`switch` – code jump for equality to constant expressions

`do while` – Loop at least once

`static` – (outside function) does not export symbol outside C module

`static` – (inside function) allocates persistent memory of a variable

`void` – non scalar type

`void *` – pointer to non scalar type

Control Flow Syntax:

- `if ( ... ) ... [else if ( ... ) ...] [else ...]`
- `while ( ... ) ...`
- `do ... while ( ... );`
- `for ( ... ; ... ; ... ) ...`
- `switch ( ... ) { case x: ... [break;] ... default: ... }`

Arithmetic Operators: `+` `-` `*` `/`

Logical Operators: `!` `&&` `||` `==` `!=` `>=` `<=` `<` `>`

Bitwise Operators: `&` `|` `^` `«` `»`

Assignment Operators: `=` `+=` `*=` `-=` `/=` `?:`

Useful functions:

Memory

`void *malloc(size_t size)` - allocates `size` bytes and returns a pointer to the allocated memory.

`void free(void *ptr)` - frees the memory space pointed to by `ptr`

`void *calloc(size_t nmemb, size_t size)` - allocates memory for an array of `nmemb` elements of `size` bytes each and returns a pointer to the allocated memory. The memory is set to zero.

`void *realloc(void *ptr, size_t size)` - changes the size of the memory block pointed to by `ptr` to `size` bytes. The contents will be unchanged in the range from the start of the region up to the minimum of the old and new sizes. If the new size is larger than the old size, the added memory will not be initialized. If `ptr` is `NULL`, then the call is equivalent to `malloc(size)`, for all values of `size`; if `size` is equal to zero, and `ptr` is not `NULL`, then the call is equivalent to `free(ptr)`. Unless `ptr` is `NULL`, it must have been returned by an earlier call to `malloc()`, `calloc()`, or `realloc()`. If the area pointed to was moved, a `free(ptr)` is done.

`void *memcpy(void *dest, const void *src, size_t n)` - copies `n` bytes from memory area `src` to memory area `dest`. The memory areas must not overlap.

`void *memset(void *s, int c, size_t n)` - fills the first `n` bytes of the memory area pointed to by `s` with the constant byte `c`. Returns a pointer to the memory area `s`.

`int strncmp(const char *s1, const char *s2, size_t n);` – return 0 if `s1` is equal to `s2`, < 0 if `s1` smaller, or > 0 if `s2` smaller. Compares at most `n` characters.

`char *strcpy(char *dest, const char *src);` – copies a string from `src` to `dst`.

`char *strncpy(char *dest, const char *src, size_t n);` – copies a string from `src` to `dst` for at most `n` characters.

`size_t strlen(const char *s);` – Calculate length of C string, excluding terminating NULL byte

`int sscanf(const char *str, const char *format, ...);` – scan input string `str` according to string `format`. Conversions of format string are stored in the location pointed to by the pointer arguments that follow `format`. `char *gets(char *str);` – get a string from standard input.

C format string

`%c` – character

`%256s` – truncation of C string

`%d` – (optionally) signed integer

`%f` – single precision float

`%s` – C string

`%.3f` – single precision float to 3 decimal places

`void qsort(void *base, size_t nmemb, size_t size,`

`int (*compar)(const void *, const void *) );` – sorts an array with `nmemb` elements of size `size`.

The `base` argument points to the start of the array. The contents of the array are sorted in ascending order according to a comparison function pointed to by `compar`, which is called with two arguments that point to the objects being compared. `compar` function must return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second.

File

`FILE *fopen(const char *pathname, const char *mode)` – mode `r` Read, `w` Write, `a` Append

`int fclose(FILE *stream)` – Closes the file

`char *fgets(char *buf, int sz, FILE *fp)` – Reads all input up until a new line or `sz-1` chars

`int fflush(FILE *stream)` – Writes data stored in the buffer to the file immediately

`ssize_t read(int fd, void *buf, size_t count);` – attempts to read up to `count` bytes from file descriptor `fd` into the buffer starting at `buf`.

`ssize_t write(int fd, const void *buf, size_t count);` – writes up to `count` bytes from the buffer pointed to by `buf` to the file referred to by the file descriptor `fd`.

`int select(int nfds, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, struct timeval *timeout);` – monitor multiple file descriptors, waiting until one or more of the file descriptors become "ready" for some class of I/O operation (e.g., input possible). A file descriptor is considered ready if it is possible to perform a corresponding I/O operation.

`void FD_CLR(int fd, fd_set *set);` – clear single descriptor descriptor from set

`int FD_ISSET(int fd, fd_set *set);` – check single descriptor active from set

```
void FD_SET(int fd, fd_set *set); - set single descriptor active from set
void FD_ZERO(fd_set *set); - zero entire descriptor set
int epoll_create(int size); - returns a file descriptor referring to the new epoll instance.
int epoll_wait(int epfd, struct epoll_event *events, int maxevents, int timeout); - waits
for events on the epoll instance by the file descriptor epfd. The buffer pointed to by events is used to
return information from the ready list about file descriptors that have events available.
```

Process

`pid_t fork(void)` - creates a new process by duplicating the calling process. On success, the PID of the child process is returned in the parent.

`int execv(const char *pathname, char *const argv[]);` - replaces the current process image with a new process image. The `const char *arg` and subsequent ellipses can be thought of as `arg0`, `arg1`, ..., `argn`. Together they describe a list of one or more pointers to NULL-terminated strings that represent the argument list available to the executed program.

```
int posix_spawn(pid_t *pid, const char *path,
                const posix_spawn_file_actions_t *file_actions,
                const posix_spawnattr_t *attrp,
                char *const argv[],
                char *const envp[]);
```

Creates a new child process that executes a specified file (the equivalent of `fork()` followed by `exec()`).

```
int posix_spawn_file_actions_init(posix_spawn_file_actions_t*);
int posix_spawn_file_actions_destroy(posix_spawn_file_actions_t*);
int posix_spawn_file_actions_addclose(posix_spawn_file_actions_t*, int fd);
int posix_spawn_file_actions_adddup2(posix_spawn_file_actions_t*, int fd, int newfd);
int posix_spawn_file_actions_addopen(posix_spawn_file_actions_t*, int fildes,
                                     const char *restrict path, int oflag, mode_t mode);
```

`int pipe(int fildes[2]);` - creates a pipe (an object that allows unidirectional data flow) and allocates a pair of file descriptors. The first descriptor connects to the read end of the pipe; the second connects to the write end.

`pid_t wait(int *wstatus);` - wait for state changes in a child of the calling process, and obtain information about the child whose state has changed.

`pid_t waitpid(pid_t pid, int *wstatus, int options);` - suspends execution of the calling thread until a child specified by `pid` argument has changed state.

`int kill(pid_t pid, int sig);` - send any signal to any process group or process. On success (at least one signal was sent), zero is returned. On error, -1 is returned, and `errno` is set appropriately.

```
typedef void (*sighandler_t)(int);
```

`sighandler_t signal(int signum, sighandler_t handler);` - sets the disposition of the signal `signum` to `handler`, which is either `SIG_IGN`, `SIG_DFL`, or the address of a programmer-defined function (a "signal

handler"). Returns the previous value of the signal handler, or `SIG_ERR` on error. In the event of an error, `errno` is set to indicate the cause.

`int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);` - change the action taken by a process on receipt of a specific signal. The `sigaction` structure is:

```
struct sigaction {
    void      (*sa_handler)(int);
    void      (*sa_sigaction)(int, siginfo_t *, void *);
    sigset_t   sa_mask;
    int        sa_flags;
    void      (*sa_restorer)(void);
};
```

`void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);`

Creates a new mapping in the virtual address space of the calling process. The starting address for the new mapping can be specified in `addr`.

The `prot` argument specifies permissions, such as `PROT_READ`, `PROT_WRITE`, `PROT_EXEC`, `PROT_NONE`

The `flags` argument specifies options, such as `MAP_ANONYMOUS` for memory not backed by any file and `MAP_SHARED` if the mapping will be shared with other processes.

`int munmap(void *addr, size_t length);` Deletes the mappings for the specified address range

Semaphore

`int sem_init(sem_t *sem, int pshared, unsigned int value);` - initializes the unnamed semaphore at the address pointed to by `sem`. The `value` argument specifies the initial value for the semaphore. Returns 0 on success; on error, -1 is returned, and `errno` is set to indicate the error.

`int sem_destroy(sem_t *sem);` - destroys the unnamed semaphore at the address pointed to by `sem`. Returns 0 on success; on error, -1 is returned, and `errno` is set to indicate the error.

`int sem_wait(sem_t *sem);` - decrements (locks) the semaphore pointed to by `sem`. If the semaphore's value is greater than zero, then the decrement proceeds, and the function returns, immediately. If the semaphore currently has the value zero, then the call blocks until either it becomes possible to perform the decrement. Returns 0 on success, otherwise -1 on error and semaphore is unchanged. `int sem_trywait(sem_t *sem);` - is the same as `sem_wait()`, except that if the decrement cannot be immediately performed, then call returns an error (`errno` set to `EAGAIN`) instead of blocking.

`int sem_post(sem_t *sem);` - increments (unlocks) the semaphore pointed to by `sem`. Returns 0 on success; on error, the value of the semaphore is left unchanged, -1 is returned, and `errno` is set.

`int sem_getvalue(sem_t *sem, int *sval);` - places the current value of the semaphore pointed to `sem` into the integer pointed to by `sval`.

Thread functions (<pthread.h>):

`int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void *), void *arg);` - starts a new thread in the calling process. The new thread starts execution

by invoking `start_routine()`; `arg` is passed as the sole argument of `start_routine()`.

`int pthread_join(pthread_t thread, void **retval);` - waits for the thread specified by `thread` to terminate. On success, returns 0; on error, it returns an error number.

Thread mutex

`int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *mutexattr);` - initializes the mutex object pointed to by `mutex` according to the mutex attributes specified in `mutexattr`.

If `mutexattr` is NULL, default attributes are used instead.

`int pthread_mutex_destroy(pthread_mutex_t *mutex);` - destroys a mutex object, freeing the resources it might hold. The mutex must be unlocked on entrance. Returns 0 on success and a non-zero error code on error.

`int pthread_mutex_lock(pthread_mutex_t *mutex);` - locks the given mutex. If the mutex is currently unlocked, it becomes locked and owned by the calling thread, and `pthread_mutex_lock` returns immediately. If the mutex is already locked by another thread, `pthread_mutex_lock` suspends the calling thread until the mutex is unlocked. Returns 0 on success and a non-zero error code on error.

`int pthread_mutex_trylock(pthread_mutex_t *mutex);` - behaves identically to `pthread_mutex_lock`, except that it does not block the calling thread if the mutex is already locked by another thread (or by the calling thread in the case of a “fast” mutex). Instead, `pthread_mutex_trylock` returns immediately with the error code EBUSY. Returns 0 on success and a non-zero error code on error.

`int pthread_mutex_unlock(pthread_mutex_t *mutex);` - unlocks the given mutex. The mutex is assumed to be locked and owned by the calling thread on entrance to `pthread_mutex_unlock`. If the mutex is of the “fast” kind, `pthread_mutex_unlock` always returns it to the unlocked state. If it is of the “recursive” kind, it decrements the locking count of the mutex (number of `pthread_mutex_lock` operations performed on it by the calling thread), and only when this count reaches zero is the mutex actually unlocked. Returns 0 on success and a non-zero error code on error.

Thread condition

`int pthread_cond_init(pthread_cond_t *cond, pthread_condattr_t *cond_attr);` - Initializes the condition variable `cond`, using the condition attributes specified in `cond_attr`, or default attributes if `cond_attr` is NULL. Returns 0 on success and a non-zero error code on error.

`int pthread_cond_signal(pthread_cond_t *cond);` - restarts one of the threads that are waiting on the condition variable `cond`. If no threads are waiting on `cond`, nothing happens. If several threads are waiting on `cond`, exactly one is restarted, but it is not specified which. Returns 0 on success and a non-zero error code on error.

`int pthread_cond_broadcast(pthread_cond_t *cond);` - restarts all the threads that are waiting on the condition variable `cond`. Nothing happens if no threads are waiting on `cond`. Returns 0 on success and a non-zero error code on error.

`int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);` - atomically unlocks the mutex (as per `pthread_unlock_mutex`) and waits for the condition variable `cond` to be signaled. The thread execution is suspended and does not consume any CPU time until the condition variable is signaled. The mutex must be locked by the calling thread on entrance to `pthread_cond_wait`. Before returning to the calling thread, `pthread_cond_wait` re-acquires mutex (as per `pthread_mutex_lock`). Returns 0 on success and a non-zero error code on error.

`int pthread_cond_timedwait(pthread_cond_t *cond, pthread_mutex_t *mutex, const struct timespec *abstime);` - atomically unlocks mutex and waits on `cond`, as `pthread_cond_wait` does, but it also bounds the duration of the wait. If `cond` has not been signaled within the amount of time specified by `abstime`, the mutex `mutex` is re-acquired and `pthread_cond_timedwait` returns the error `ETIMEDOUT`. The `abstime` parameter specifies an absolute time, with the same origin as `time(2)` and `gettimeofday(2)`: an `abstime` of 0 corresponds to 00:00:00 GMT, January 1, 1970. Returns 0 on success and a non-zero error code on error.

`int pthread_cond_destroy(pthread_cond_t *cond);` - destroys a condition variable, freeing the resources it might hold. No threads must be waiting on the condition variable on entrance to `pthread_cond_destroy`. Returns 0 on success and a non-zero error code on error.

Note: timed variants of threaded functions are not permitted where specified

BARRIER

`int pthread_barrier_init(pthread_barrier_t *restrict barrier, const pthread_barrierattr_t *restrict attr, unsigned count);` - allocate any resources required to use the barrier referenced by `barrier` and shall initialize the barrier with attributes referenced by `attr`. If `attr` is `NULL`, the default barrier attributes shall be used. The results are undefined if `pthread_barrier_init()` is called when any thread is blocked on the barrier (that is, has not returned from the `pthread_barrier_wait()` call). The results are undefined if a barrier is used without first being initialized. The results are undefined if `pthread_barrier_init()` is called specifying an already initialized barrier. The `count` argument specifies the number of threads that must call `pthread_barrier_wait()` before any of them successfully return from the call. The value specified by `count` must be greater than zero. If the `pthread_barrier_init()` function fails, the barrier shall not be initialized and the contents of `barrier` are undefined. Only the object referenced by `barrier` may be used for performing synchronization. The result of referring to copies of that object in calls to `pthread_barrier_destroy()` or `pthread_barrier_wait()` is undefined. Returns zero; otherwise, an error number shall be returned to indicate the error.

`int pthread_barrier_destroy(pthread_barrier_t *barrier);` - destroy the barrier referenced by `barrier` and release any resources used by the barrier. The effect of subsequent use of the barrier is undefined until the barrier is reinitialized by another call to `pthread_barrier_init()`. Returns zero; otherwise, an error number shall be returned to indicate the error.

`int pthread_barrier_wait(pthread_barrier_t *barrier);` - synchronize participating threads at the

barrier referenced by **barrier**. The calling thread shall block until the required number of threads have called `pthread_barrier_wait()` specifying the barrier. Returns `PTHREAD_BARRIER_SERIAL_THREAD` for a single (arbitrary) thread synchronized at the barrier and zero for each of the other threads. Otherwise, an error number shall be returned to indicate the error.